

Writeup for the Submission of MLSys 2026 Competition

Ryang Sohn

April 25, 2026

1 Introduction and Approach

This writeup outlines the overall approach we have taken to implement a computation graph optimizer.

The overall approach was inspired by two prior works: Korch[1] and Welder[2]. From Korch, we have adopted a kernel selection strategy using BLP (Binary Linear Programming) to allow selecting schedules with recomputations of some operations. Also, we have implemented a selection algorithm to determine retained tensors based on the `GraphConnecting` algorithm introduced in Welder.

The overall flow of the optimizer is as follows:

1. The optimizer enumerates convex subgraphs of the whole graph.
2. Convex subgraphs are benchmarked using the performance model.
3. Using the costs obtained from the benchmark, the optimizer selects the subgraphs using BLP.
4. The selected subgraphs are further optimized by selecting tensors to retain.

1.1 Convex Subgraph Enumeration

Korch defines the notion of “convex subgraph” as follows: a subgraph $G' = (V', E')$ of a DAG $G = (V, E)$ is *convex* if for every directed path in G whose endpoints both lie in V' , every intermediate vertex on that path is also in V' . This guarantees that the subgraph can be scheduled as a single fused kernel without splitting an operation away from any of its transitive dependencies that lie “inside” the kernel boundary.

We adopt this definition and use convex subgraphs as units of kernel selection in the next step, subgraph selection. The problem constraints dictate that all subgraphs must be connected, so we make sure that all enumerated convex subgraphs are connected (i.e., each subgraph has only one component).

Rather than the pairwise-difference enumeration described by Korch, we use a seed-and-grow procedure: for each operation $v \in V$, we initialize $S = \{v\}$ and recursively try to add each frontier neighbor w (a predecessor or successor of some node in S). For convexity, we maintain the ancestor and descendant bitsets $\text{anc}(S)$ and $\text{desc}(S)$ precomputed once via topological sweeps; a candidate $S \cup \{w\}$ is convex iff $\text{anc}(S \cup \{w\}) \cap \text{desc}(S \cup \{w\}) \subseteq S \cup \{w\}$. Visited subsets are deduplicated by a hash of their bitset, and growth halts at a fixed maximum fusion size, since fusion benefit plateaus once the working set exceeds fast memory.

1.2 Benchmarking Convex Subgraphs

For each enumerated subgraph, the optimizer searches $(\mathbf{w}, \mathbf{h}, \mathbf{k})$ tile sizes in parallel and picks the one minimizing input and output memory traffic, preferring larger tiles to break ties. Candidate spatial tiles are divisors of each output’s width (resp. height) up to native granularity, intersected across all outputs. Candidate reduction tiles are divisors of the smallest matmul K that also divide every other matmul’s K , capped by the native granularity. Feasibility is checked by projecting per-op iteration dimensions $(m_O, n_O, k_O^{\text{split}})$ backward through the subgraph in reverse topological order — a boundary-producing op takes the global granule, an interior op feeding a pointwise consumer inherits its dimensions, and an interior op feeding a matmul consumer contracts to that consumer’s LHS row strip or RHS column strip — and summing LHS strips, RHS strips, pointwise tiles, and output accumulators into a conservative peak working set that must fit in fast

memory minus retained-tensor reservation. Split- k triples are also rejected when a fused pointwise prologue’s tile would not cover the consumer matmul’s full reduction.

After choosing tile size, the performance model determines the cost of the subgraph using the rules defined in the problem statement, also in parallel.

1.3 Subgraph Selection

The benchmarked subgraphs are then fed into the kernel selector. The kernel selector uses BLP to determine a set of optimal kernels to execute. Constraints are added to the BLP problem instance to ensure that all output tensors are computed and dependencies are not violated.

In this implementation, we used the `good_lp` crate to interface with the HiGHS LP solver, which is suitable because it allows linking statically to the binary.

1.4 Retention and Traversal

The selected kernels are then optimized further by retaining some of the tensors in fast memory. When a tensor is retained in fast memory, the producer and the consumers of the tensor must be in different subgraphs. We implemented a memory-level selection algorithm inspired by Welder’s `GraphConnecting` algorithm. In this implementation, fast memory is treated as “higher level” than ephemeral memory, and we added beam search to consider more candidates than a greedy algorithm.

2 Results

We evaluated the optimizer on the five public benchmarks, comparing the total latency it produces against a naive baseline that schedules each operation as its own subgraph at the largest feasible tile. Latencies are reported in the cost model’s native units; the speedup column is the naive latency divided by the optimized latency.

Benchmark	Ops	Naive latency	Optimized latency	Speedup
<code>mlsys-1</code>	5	576,716.80	576,716.80	1.00×
<code>mlsys-5</code>	19	1,459,268.27	1,160,329.60	1.26×
<code>mlsys-9</code>	32	80,530,636.80	76,373,155.84	1.05×
<code>mlsys-13</code>	63	22,165,913.60	22,031,564.80	1.01×
<code>mlsys-17</code>	103	5,081,850.88	4,983,971.84	1.02×

The optimizer never regresses against the baseline. The largest gain is on `mlsys-5`, whose graph has enough fusible pointwise neighborhoods to amortize matmul output writes; on `mlsys-1`, all five operations are already memory-bound at their native tile, so no fusion improves on the per-op schedule.

3 Discussion

This challenge was a great opportunity to explore deep learning system optimization, specifically kernel fusion and graph compilers. Given more time, we would have explored non-convex kernel mapping as presented in FlashTensor.

References

- [1] Muyan Li, Peng Wang, Ying Lin, Yuanjun Pang, Tao Xu, and Zhihao Jia. Optimal kernel orchestration for tensor programs with korch. In *Proceedings of ASPLOS*, 2024.
- [2] Yining Shi et al. Welder: Scheduling deep learning memory access via tile-graph. In *OSDI*, 2023.